CS3000 Practice Quiz 5A — Answer Key

Coin change and LCS

Notation follows the class notes on dynamic programming: $C(v)$ (coin change), $K(i, w)$ (0/1 knapsack), $L(i, j)$ (LCS), $D(i, j)$ (edit distance); strings and items are 1-indexed. Every table below was computed by a program and checked against brute force. Shaded cells mark the traceback path. Point splits imitate the Gradescope rubrics used on Quizzes 1–4 (each question 5 pts, 10 total).

Q1. Coin change with $d = \{1, 4, 6\}$, $v = 9$

(a) Recurrence.

$$C(v) = \begin{cases} 0 & v = 0 \\ \min\limits_{i:\, d_i \le v} \left(1 + C(v - d_i)\right) & v > 0 \end{cases} \qquad (C(v) = \infty \text{ if the min is over an empty set}).$$

Reason: any solution for $v > 0$ uses some last coin d_i; the rest must be an optimal way to make $v - d_i$.

(b) Table.

v	0	1	2	3	4	5	6	7	8	9
$C(v)$	0	1	2	3	1	2	1	2	2	3

Sample entries: $C(5) = \min(1+C(4),\ 1+C(1)) = \min(2, 2) = 2$; $C(8) = \min(1+C(7), 1+C(4), 1+C(2)) = \min(3, 2, 3) = 2$; $C(9) = \min(1 + C(8), 1 + C(5), 1 + C(3)) = \min(3, 3, 4) = 3$.

(c) Greedy vs. optimal. Greedy takes 6, then $1 + 1 + 1$: **4 coins**. The optimum is $C(9) = \mathbf{3}$ coins: $4 + 4 + 1$. Greedy fails because committing to the 6 leaves a remainder (3) that only 1-coins can make.

(d) Returning the coins. Store the last coin chosen for each amount, then walk back.

```
function COIN-CHANGE-COINS (denominations, v):
    C = new array [0..v];  last = new array [0..v]
    C[0] = 0
    for a from 1 to v+1:          // index v+1 non-inclusive
        C[a] = infinity
        for each coin d in denominations:
            if d <= a and 1 + C[a - d] < C[a]:
                C[a] = 1 + C[a - d]
                last[a] = d
    if C[v] == infinity: return "impossible"
    coins = empty list;  a = v
    while a > 0:
        append last[a] to coins
        a = a - last[a]
    return C[v], coins
```

Running time $\Theta(kv)$ (the fill is v iterations of a k-coin loop; the walk back is $O(v)$). Extra space $\Theta(v)$ for C and `last`. For $v = 9$ this returns e.g. $[1, 4, 4]$ (any order of $\{4, 4, 1\}$ is correct).

Rubric (5 pts).

+1 pt Correct recurrence with base case $C(0) = 0$ (and ∞ when no coin fits).

+1.5 pts Table fully correct (-0.5 per wrong entry, min 0).

+1 pt Greedy gives 4 coins ($6 + 1 + 1 + 1$); optimal is 3 coins ($4 + 4 + 1$).

+1.5 pts Pseudocode records the choice and walks back correctly; $\Theta(kv)$ time, $\Theta(v)$ space.

Common mistakes: forgetting $C(0) = 0$ or the ∞ case; writing the knapsack-style "use each coin once" recurrence; claiming greedy is optimal.

Q2. LCS of $X =$ `TACGA`, $Y =$ `ACTGAC`

(a) Table (recurrence: $L(i, j) = 0$ if $i = 0$ or $j = 0$; $L(i-1, j-1) + 1$ if $X[i] = Y[j]$; else $\max(L(i-1, j), L(i, j-1))$).

$X_i \downarrow$ $Y_j \rightarrow$	0	1 (A)	2 (C)	3 (T)	4 (G)	5 (A)	6 (C)
0	0	0	0	0	0	0	0
1 (T)	**0**	0	0	1	1	1	1
2 (A)	0	**1**	1	1	1	2	2
3 (C)	0	1	**2**	**2**	2	2	3
4 (G)	0	1	2	2	**3**	3	3
5 (A)	0	1	2	2	3	**4**	**4**

(b) $L(5, 6) = \mathbf{4}$. Traceback from $(5, 6)$: on a match take the character and move diagonally; otherwise move to the larger of up/left (up on ties). The path (shaded) gives the LCS `ACGA`, which is the only LCS of these strings.

(c) Traceback pseudocode.

```
function LCS-TRACEBACK (L, X, Y):
    i = length(X);  j = length(Y)
    out = empty list
    while i > 0 and j > 0:
        if X[i] == Y[j]:
            append X[i] to out
            i = i - 1;  j = j - 1
        else if L[i-1, j] >= L[i, j-1]:
            i = i - 1
        else:
            j = j - 1
    return reverse(out)
```

Each step decreases i or j (or both), so it runs in $O(m + n)$ time.

Rubric (5 pts).

+2 pts Table fully correct (-0.5 per wrong row).

+1 pt Length 4 and a valid LCS (`ACGA`) recovered by a correct path.

+1.5 pts Traceback pseudocode: diagonal on match, larger neighbor otherwise, reverses output.

+0.5 pts Running time $O(m + n)$.

Common mistakes: moving diagonally on a mismatch; reading the LCS front-to-back instead of reversing it; forgetting the row/column of zeros.